Relatório de Desenvolvimento - ‘Fase 5’

Desenvolvimento de Software — Prof. Dr. Marcelo Soares Pimenta — INF01120

Davi Santos	00599530
Gabriel Schons	00342020
Gabriel Copatti	00342808
Julio Augusto de Castilhos Borges	00590537

Link para o repositório do trabalho: https://github.com/Davi-FSantos/INF-01120_26-1

Introdução e Suposições de Projeto

Este relatório documenta a entrega final da **Music Machine**, englobando todo o histórico de requisitos e classes das Fases 1 e 2, bem como as ações corretivas de refatoração tomadas a partir da revisão de código realizada pelo Grupo Revisor 4.

Para a realização do trabalho, foram estabelecidas as seguintes suposições:

1. **Bancos de Timbres (SoundFonts)**: O motor de síntese FluidSynth exige um arquivo de SoundFont (`.sf2` ou `.sf3`) General MIDI. O sistema procura automaticamente em diretórios padrões de áudio do Linux. Caso não encontre, provê uma caixa de diálogo interativa (`QFileDialog`) para indicação manual.
2. **Reserva do Canal MIDI 9**: O canal MIDI de índice 9 é reservado por especificação para percussão. Para evitar conflitos, o mapeamento de canais pula o índice 9 e cicla entre os demais.
3. **Limites de BPM**: O BPM é controlado globalmente, limitado no intervalo de 40 a 1200 batidas por minuto.

Lista de Requisitos do Sistema

Requisitos Funcionais - Fase 1 (Original)

1. Fornecer um campo de entrada de texto para inserir ou colar sequências.
2. Mapear caracteres específicos para instruções musicais definidas.
3. Permitir configurar parâmetros iniciais (BPM, instrumento, oitava, volume).
4. Sintetizar e tocar a sequência de áudio resultante no dispositivo padrão.
5. Exibir interface gráfica com controles de reprodução.
6. Permitir Iniciar, Pausar e Reiniciar a reprodução.
7. Validar entrada e fornecer feedback sobre caracteres inválidos.
8. Manter o estado atual dos parâmetros musicais do sistema.

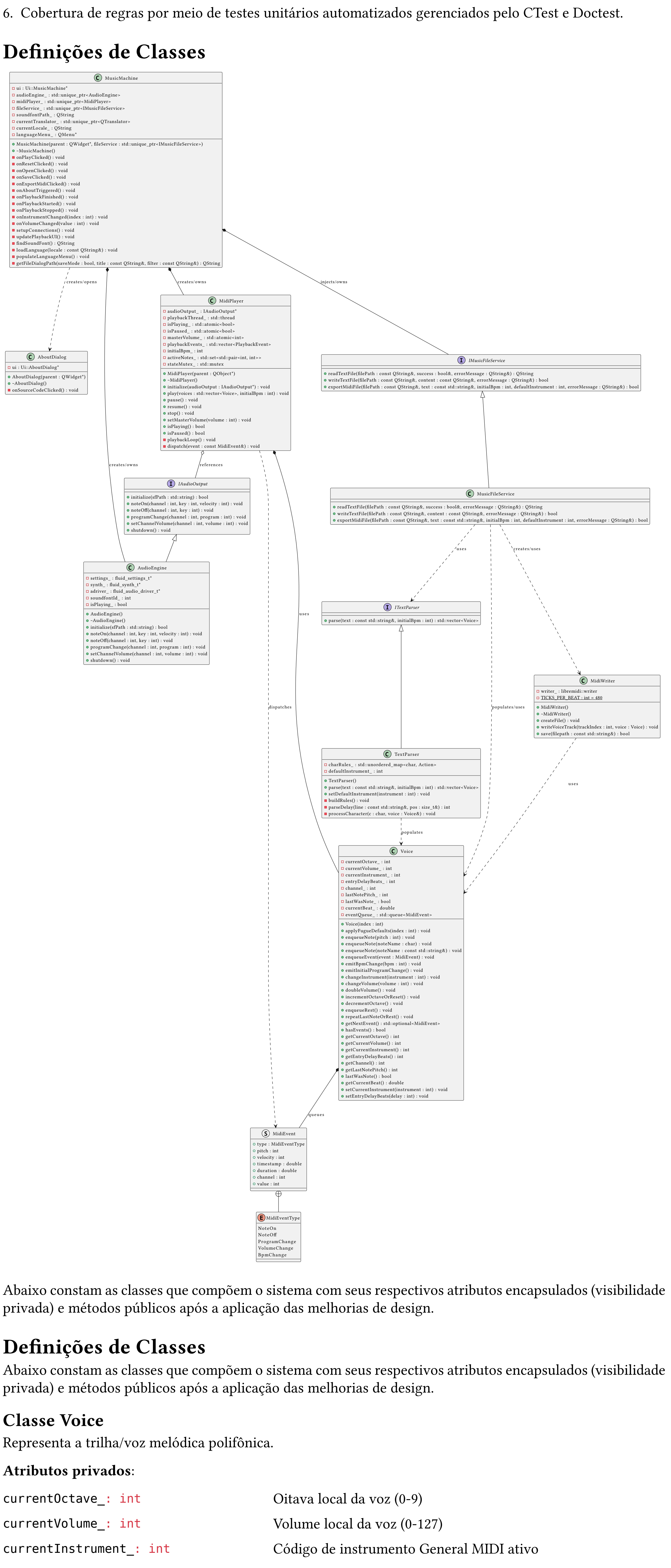
Requisitos Funcionais - Fase 2 (Atualizados)

1. Área de edição de texto para visualização e edição direta.
2. Carregamento de arquivos `.txt` e gravação do texto modificado de volta ao disco.
3. Tratamento de cada linha de texto como uma voz musical independente ($V_0, V_1, V_2, \ldots$) executadas de forma polifônica e simultânea.
4. Mapeamento de caracteres locais para regras de notas, volumes, instrumentos e oitavas por voz.
5. Suporte a atrasos iniciais por linha usando o formato `[n]`.
6. Alteração global de BPM em tempo real durante a reprodução via símbolos > e <.
7. Exportação da partitura processada para arquivo MIDI padrão (`.mid`) multilhas.
8. Síntese com FluidSynth e SoundFont em tempo real.
9. Execução da síntese de som de forma assíncrona em thread dedicada para manter a GUI responsiva.
10. Controles visuais de Play, Pause, Stop e volume mestre.

Requisitos Não Funcionais

1. Aderência rígida a princípios de design orientado a objetos (SOLID).
2. Uso de linguagem C++ padrão 26 para desenvolvimento principal.
3. Interface gráfica desenvolvida com a biblioteca Qt6 e Qt Designer.
4. Gerenciamento e compilação do projeto multiplataforma via Xmake.
5. Controle de versão via Git.
6. Cobertura de regras por meio de testes unitários automatizados gerenciados pelo CTest e Doctest.

Definições de Classes


MusicMachine
ui : Ui::MusicMachine*
audioEngine_ : std::unique_ptr<AudioEngine>
midiPlayer_ : std::unique_ptr<MidiPlayer>
fileService_ : std::unique_ptr<IMusicFileService>
soundfontPath_ : QString
currentTranslator_ : std::unique_ptr<QTranslator>
currentLocale_ : QString
languageMenu_ : QMenu*
MusicMachine(parent : QWidget*, fileService : std::unique_ptr<IMusicFileService>)
~MusicMachine()
onPlayClicked() : void
onResetClicked() : void
onOpenClicked() : void
onSaveClicked() : void
onExportMidiClicked() : void
onAboutTriggered() : void
onPlaybackFinished() : void
onPlaybackStarted() : void
onPlaybackStopped() : void
onInstrumentChanged(index : int) : void
onVolumeChanged(value : int) : void
setupConnections() : void
updatePlaybackUI() : void
findSoundFont() : QString
loadLanguage(locale : const QString&) : void
populateLanguageMenu() : void
getFileDialogPath(saveMode : bool, title : const QString&, filter : const QString&) : QString
creates/opens
creates/owns
injects/owns
AboutDialog
ui : Ui::AboutDialog*
AboutDialog(parent : QWidget*)
~AboutDialog()
onSourceCodeClicked() : void
MidiPlayer
audioOutput_ : IAudioOutput*
playbackThread_ : std::thread
isPlaying_ : std::atomic<bool>
isPaused_ : std::atomic<bool>
masterVolume_ : std::atomic<int>
playbackEvents_ : std::vector<PlaybackEvent>
initialBpm_ : int
activeNotes_ : std::set<std::pair<int, int>>
stateMutex_ : std::mutex
MidiPlayer(parent : QObject*)
~MidiPlayer()
initialize(audioOutput : IAudioOutput*) : void
play(voices : std::vector<Voice>, initialBpm : int) : void
pause() : void
resume() : void
stop() : void
setMasterVolume(volume : int) : void
isPlaying() : bool
isPaused() : bool
playbackLoop() : void
dispatch(event : const MidiEvent&) : void
references
creates/owns
IAudioOutput
initialize(sfPath : std::string) : bool
noteOn(channel : int, key : int, velocity : int) : void
noteOff(channel : int, key : int) : void
programChange(channel : int, program : int) : void
setChannelVolume(channel : int, volume : int) : void
shutdown() : void
AudioEngine
settings_ : fluid_settings_t*
synth_ : fluid_synth_t*
adriver_ : fluid_audio_driver_t*
soundfontId_ : int
isPlaying_ : bool
AudioEngine()
~AudioEngine()
initialize(sfPath : std::string) : bool
noteOn(channel : int, key : int, velocity : int) : void
noteOff(channel : int, key : int) : void
programChange(channel : int, program : int) : void
setChannelVolume(channel : int, volume : int) : void
shutdown() : void
IMusicFileService
readTextFile(filePath : const QString&, success : bool&, errorMessage : QString&) : QString
writeTextFile(filePath : const QString&, content : const QString&, errorMessage : QString&) : bool
exportMidiFile(filePath : const QString&, text : const std::string&, initialBpm : int, defaultInstrument : int, errorMessage : QString&) : bool
MusicFileService
readTextFile(filePath : const QString&, success : bool&, errorMessage : QString&) : QString
writeTextFile(filePath : const QString&, content : const QString&, errorMessage : QString&) : bool
exportMidiFile(filePath : const QString&, text : const std::string&, initialBpm : int, defaultInstrument : int, errorMessage : QString&) : bool
uses
creates/uses
populates/uses
ITextParser
parse(text : const std::string&, initialBpm : int) : std::vector<Voice>
MidiWriter
writer_ : libremidi::writer
TICKS_PER_BEAT : int = 480
MidiWriter()
~MidiWriter()
createFile() : void
writeVoiceTrack(trackIndex : int, voice : Voice) : void
save(filepath : const std::string&) : bool
uses
TextParser
charRules_ : std::unordered_map<char, Action>
defaultInstrument_ : int
TextParser()
parse(text : const std::string&, initialBpm : int) : std::vector<Voice>
setDefaultInstrument(instrument : int) : void
buildRules() : void
parseDelay(line : const std::string&, pos : size_t&) : int
processCharacter(c : char, voice : Voice&) : void
populates
dispatches
uses
Voice
currentOctave_ : int
currentVolume_ : int
currentInstrument_ : int
entryDelayBeats_ : int
channel_ : int
lastNotePitch_ : int
lastWasNote_ : bool
currentBeat_ : double
eventQueue_ : std::queue<MidiEvent>
Voice(index : int)
applyFugueDefaults(index : int) : void
enqueueNote(pitch : int) : void
enqueueNote(noteName : char) : void
enqueueNote(noteName : const std::string&) : void
enqueueEvent(event : MidiEvent) : void
emitBpmChange(bpm : int) : void
emitInitialProgramChange() : void
changeInstrument(instrument : int) : void
changeVolume(volume : int) : void
doubleVolume() : void
incrementOctaveOrReset() : void
decrementOctave() : void
enqueueRest() : void
repeatLastNoteOrRest() : void
getNextEvent() : std::optional<MidiEvent>
hasEvents() : bool
getCurrentOctave() : int
getCurrentVolume() : int
getCurrentInstrument() : int
getEntryDelayBeats() : int
getChannel() : int
getLastNotePitch() : int
lastWasNote() : bool
getCurrentBeat() : double
setCurrentInstrument(instrument : int) : void
setEntryDelayBeats(delay : int) : void
queues
MidiEvent
type : MidiEventType
pitch : int
velocity : int
timestamp : double
duration : double
channel : int
value : int
MidiEventType
NoteOn
NoteOff
ProgramChange
VolumeChange
BpmChange

Abaixo constam as classes que compõem o sistema com seus respectivos atributos encapsulados (visibilidade privada) e métodos públicos após a aplicação das melhorias de design.

Definições de Classes

Abaixo constam as classes que compõem o sistema com seus respectivos atributos encapsulados (visibilidade privada) e métodos públicos após a aplicação das melhorias de design.

Classe Voice

Representa a trilha/voz melódica polifônica.

Atributos privados:

`currentOctave_: int`	Oitava local da voz (0-9)
`currentVolume_: int`	Volume local da voz (0-127)
`currentInstrument_: int`	Código de instrumento General MIDI ativo
`entryDelayBeats_: int`	Silêncio inicial acumulado da voz
`channel_: int`	Canal MIDI físico da voz (pulando canal 9)
`lastNotePitch_: int`	Tom da última nota executada
`lastWasNote_: bool`	Sinalizador se o último evento foi nota
`currentBeat_: double`	Tempo acumulado do cursor na trilha
`eventQueue_: std::queue<MidiEvent>`	Fila de eventos musicais agendados

Métodos públicos:

`Voice(index: int)`	Construtor principal aplicando as propriedades iniciais de voz da fuga.
`applyFugueDefaults(index: int): void`	Configura oitava, volume e canal padrão com base no índice.
`enqueueNote(pitch: int): void`	Enfileira eventos de ligar/desligar nota física.
`enqueueNote(noteName: char): void`	Resolve tom dinâmico baseado na oitava atual e enfileira.
`enqueueNote(noteName: string): void`	Resolve tom de nota com alteração (ex: Eb) e enfileira.
`enqueueEvent(event: MidiEvent): void`	Insere um evento genérico na fila.
`emitBpmChange(bpm: int): void`	Adiciona evento de mudança global de BPM.
`emitInitialProgramChange(): void`	Emite mudança de instrumento padrão inicial no instante 0.
`changeInstrument(instrument: int): void`	Executa a troca de instrumento e enfileira o evento associado.
`changeVolume(volume: int): void`	Ajusta o volume interno (clamping) e enfileira evento.
`doubleVolume(): void`	Dobra o volume corrente respeitando os limites MIDI.
`incrementOctaveOrReset(): void`	Sobe oitava ou retorna à oitava padrão.
`decrementOctave(): void`	Diminui oitava local.
`enqueueRest(): void`	Insere tempo de pausa/silêncio.
`repeatLastNoteOrRest(): void`	Repete a última nota caso aplicável, senão insere silêncio.

Classe IMusicFileService / MusicFileService

Interface e implementação de serviço para isolamento de operações de persistência e conversões externas de acordo com o princípio SRP.

Métodos públicos:

`readTextFile(filePath: QString, success: bool&, errorMessage: QString&): QString`	Lê um arquivo textual contendo partituras.
`writeTextFile(filePath: QString, content: QString, errorMessage: QString&): bool`	Salva o texto corrente da interface em disco.
`exportMidiFile(filePath: QString, text: string, initialBpm: int, defaultInstrument: int, errorMessage: QString&): bool`	Invoca o parser e o gravador MIDI salvando a partitura multilhas convertida para arquivo físico.

Interface ITextParser / Classe TextParser

Responsável pela interpretação sintática e semântica do texto.

Atributos privados:

`defaultInstrument_: int`	Instrumento inicial selecionado no widget principal
`charRules_: std::unordered_map<char, RuleFunction>`	Dicionário de regras mapeando caracteres para ações nas vozes

Métodos públicos:

`parse(text: string, initialBpm: int): std::vector<Voice>`	Realiza a decomposição em linhas e constrói a coleção de vozes configuradas.
`parseDelay(line: string, pos: size_t&): int`	Varre a marcação `[n]` retornando o valor do atraso de início.
`processCharacter(character: char, voice: Voice&): void`	Despacha a regra associada ao caractere sobre a instância da voz.
`setDefaultInstrument(instrument: int): void`	Armazena o instrumento mestre inicial.

Classe MusicMachine (MainWindow)

Controlador gráfico que gerencia os widgets de interface e delega comandos de reprodução e persistência.

Atributos privados:

`ui: Ui::MusicMachine*`	Referência para interface visual carregada do XML
`audioEngine_: std::unique_ptr<AudioEngine>`	Motor FluidSynth
`midiPlayer_: std::unique_ptr<MidiPlayer>`	Motor de playback assíncrono de eventos
`fileService_: std::unique_ptr<IMusicFileService>`	Serviço de manipulação física de arquivos (DIP)
`soundfontPath_: QString`	Caminho do banco de áudio carregado

Métodos públicos:

`MusicMachine(parent: QWidget*, fileService: std::unique_ptr<IMusicFileService>)`	Injeta opcionalmente o serviço.
`getFileDialogPath(saveMode: bool, title: QString, filter: QString): QString`	Método auxiliar que unifica diálogos de busca e gravação física de arquivos.
`onPlayClicked(), onResetClicked(), onOpenClicked(), onSaveClicked(), onExportMidiClicked(), onVolumeChanged(int)`	Slots de eventos de controle de botões e seletores.

Outras Classes Auxiliares:

`AudioEngine`	Interface concreta para FluidSynth (`noteOn`, `noteOff`, `programChange`, `volume`).
`MidiPlayer`	Temporizador multithread responsável por consumir as filas de eventos em tempo real.
`MidiWriter`	Conversor de listas de eventos musicais em blocos estruturados da biblioteca `libremidi`.

Interface Gráfica (GUI)

Croqui

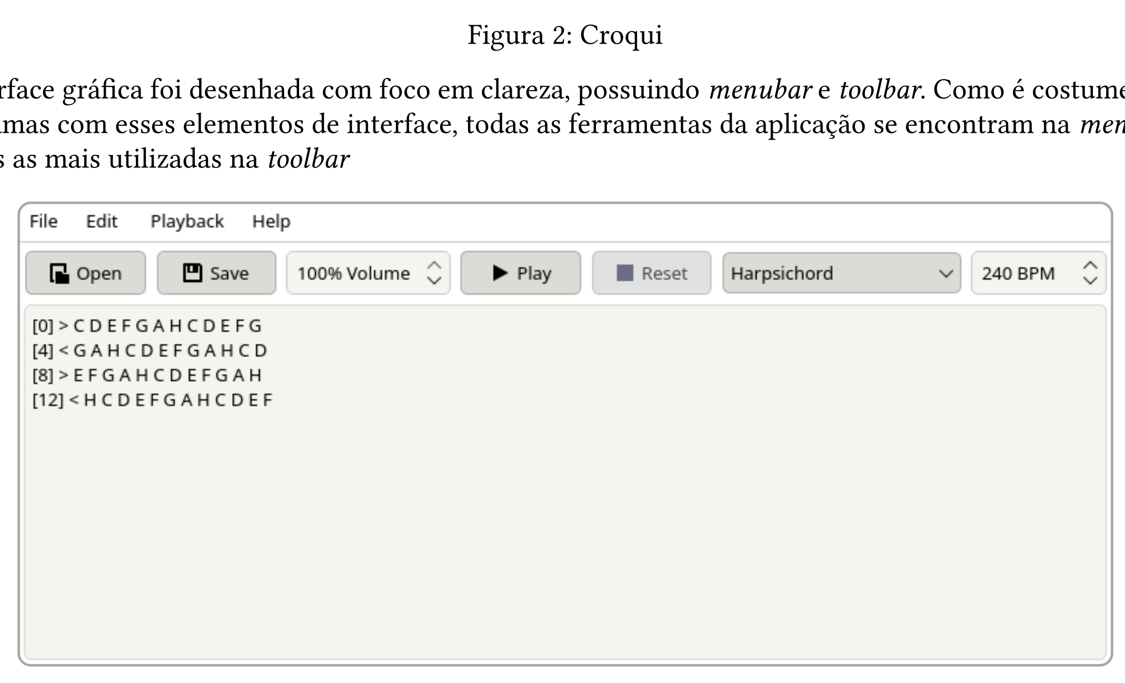
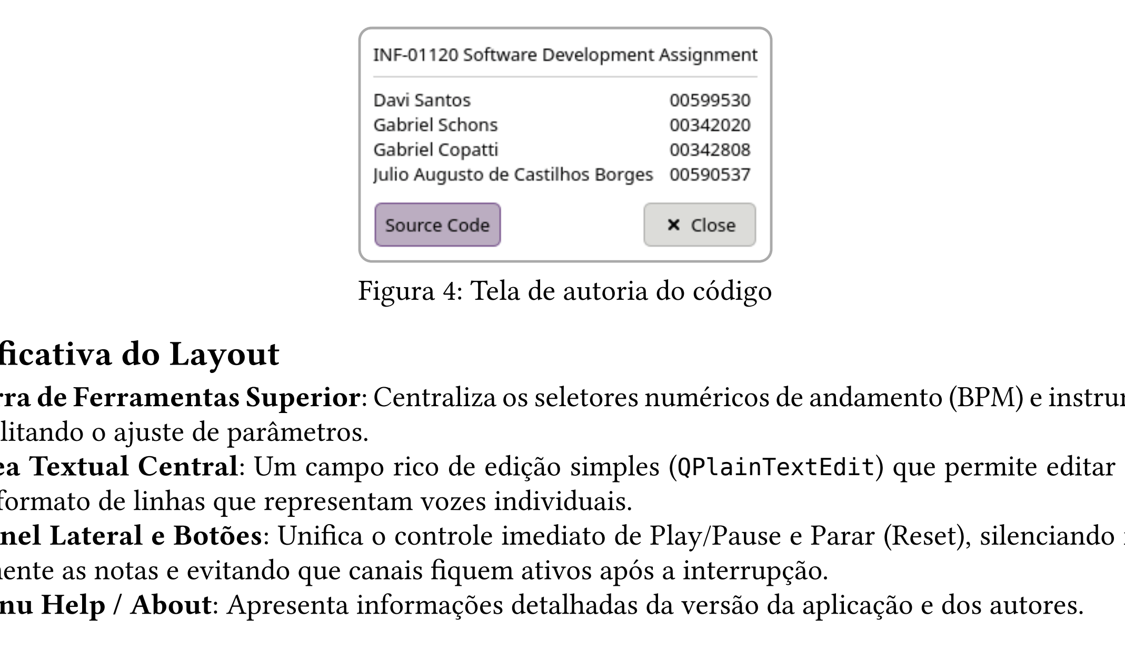

Open File
Save File
Volume
50%
Play
Reset
Instrument
Marimba
BPM
120
Text Editing

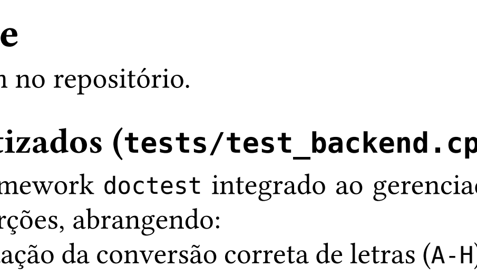
Figura 2: Croqui

A interface gráfica foi desenhada com foco em clareza, possuindo *menubar* e *toolbar*. Como é costumeiro em programas com esses elementos de interface, todas as ferramentas da aplicação se encontram na *menubar*, e apenas as mais utilizadas na *toolbar*


File
Edit
Playback
Help
Open
Save
100% Volume
Play
Reset
Harpsichord
240 BPM

```
[0] > C D E F G A H C D E F G
[4] < G A H C D E F G A H C D
[8] > E F G A H C D E F G A H
[12] < H C D E F G A H C D E F
```

Figura 3: Interface Gráfica Principal



INF-01120 Software Development Assignment

Davi Santos	00599530
Gabriel Schons	00342020
Gabriel Copatti	00342808
Julio Augusto de Castilhos Borges	00590537

Source Code
✕ Close

Figura 4: Tela de autoria do código

Justificativa do Layout

1. **Barra de Ferramentas Superior**: Centraliza os seletores numéricos de andamento (BPM) e instrumentos, facilitando o ajuste de parâmetros.
2. **Área Textual Central**: Um campo rico de edição simples (`QPlainTextEdit`) que permite editar o texto no formato de linhas que representam vozes individuais.
3. **Painel Lateral e Botões**: Unifica o controle imediato de Play/Pause e Parar (Reset), silenciando imediatamente as notas e evitando que canais fiquem ativos após a interrupção.
4. **Menu Help / About**: Apresenta informações detalhadas da versão da aplicação e dos autores.

Procedimento de Teste

Os arquivos de teste se encontram no repositório.

Testes Unitários Automatizados (`tests/test_backend.cpp`)

Os testes unitários utilizam o framework `doctest` integrado ao gerenciador do Xmake. A suíte executa 11 casos de testes cobrindo 144 asserções, abrangendo:

1. **Mapeamento de Notas**: Validação da conversão correta de letras (`A-H`) para pitches correspondentes sob diversas oitavas, e testes com clamping fora de escala (0 a 127).
2. **Notas com Acidentes**: Correção do parse de compostos como `Eb`, `Ab`, `Mb`.
3. **Atrasos e Polyphony**: Garantia de que o parser gera o tempo exato de silêncio inicial (`[n]`) em trilhas de múltiplas vozes concorrentes.
4. **Controles de Volume e Instrumentos**: Validação do aumento de volume físico e troca de patch General MIDI baseado em dígitos, vogais e marcações especiais (`!`, `,`, `;`).
5. **Aceleração Global de BPM**: Testes programáticos do sincronismo dinâmico nos comandos > e <.
6. **Controle de Repetições**: Validação da duplicação da última nota por caracteres não reconhecidos.

Testes Manuais com Arquivos Estruturados

Adicionalmente, arquivos padrões de testes são fornecidos sob o diretório `tests/` para validação e escuta via GUI:

- `fuga_bwv847.txt` e `fuga_bwv578.txt`: Para testar polifonia clássica de Bach.
- `fuga_bpm.txt`: Modificação dinâmica de andamento durante reprodução.
- `fuga_instrumentos.txt`: Alterações sucessivas de canais e patches.

Relatório de Revisão de Código e Ações Corretivas

Abaixo, detalhamos cada ponto identificado pelo Grupo Revisor 4 em seu relatório e as respectivas ações que implementamos na estrutura do sistema:

A. Princípio de Responsabilidade Única (SRP) e Inversão de Dependências (DIP) na GUI

- **Problema Apontado**: A classe visual `MusicMachine` (MainWindow) acumulava responsabilidades de leitura e gravação no disco (`QFile`/`QTextStream`) e geração de MIDI, acoplando a lógica com componentes de tela.
- **Ação Corretiva**: Criamos a interface abstrata `IMusicFileService` e a implementação concreta `MusicFileService`. Toda a manipulação de leitura, gravação e o encadeamento de exportação MIDI do `TextParser` com o `MidiWriter` foram extraídos para o serviço. O controlador visual apenas requisita a persistência via interface, permitindo mock de testes e desacoplamento completo.

B. Encapsulamento de Atributos do Modelo `Voice`

- **Problema Apontado**: Variáveis e estados fundamentais (como oitavas locais e volume corrente) estavam declarados de forma pública.
- **Ação Corretiva**: Tornamos todos os campos de controle internos privados no cabeçalho `Voice.h` e expusemos métodos getters públicos limpos (`getCurrentOctave()`, `getCurrentVolume()`, etc.), mantendo a integridade dos limites e consistência do objeto.

C. Inveja de Dados (Feature Envy) em `TextParser`

- **Problema Apontado**: A classe de parsing continha fórmulas e mutações diretas sobre os valores internos de `Voice` (ex: `voice.currentVolume = std::min(127, voice.currentVolume * 2)`), violando as boas práticas de encapsulamento comportamental.
- **Ação Corretiva**: Adicionamos métodos de ação com regras explícitas na classe `Voice`, como `doubleVolume()`, `changeInstrument(int)`, `incrementOctaveOrReset()`, e `decrementOctave()`. O processador de texto agora se limita a disparar esses métodos sob a voz correspondente, deixando que a voz gerencie sua matemática interna.

D. Eliminação de Código Duplicado na Interface

- **Problema Apontado**: Códigos estruturalmente idênticos para configuração de caixas de diálogo e tratamento de fluxos de erros físicos em `onOpenClicked()`, `onSaveClicked()` e `onExportMidiClicked()`.
- **Ação Corretiva**: Implementamos o método centralizado `getFileDialogPath(bool saveMode, const QString& title, const QString& filter)` que unifica a abertura de diálogos do Qt6, reduzindo a duplicação e facilitando o gerenciamento centralizado de erros de cancelamento ou leitura.

E. Eliminação de Números Mágicos na Conversão

- **Problema Apontado**: As conversões de semitons manuais (`+ 3` e `+ 8`) no motor de pitch MIDI eram representadas por inteiros literais jogados no meio das expressões matemáticas.
- **Ação Corretiva**: Definimos constantes estáticas explicativas (`SEMITONE_E_FLAT = 3` e `SEMITONE_A_FLAT = 8`) no modelo para documentar e organizar os cálculos matemáticos.

Números Mágicos na classe MidiWriter

Enquanto mostrávamos o código para o professor, também indentificamos que a classe `MidiWriter` poderia ter sua qualidade interna melhorada, assim:

- **Problema**: O arquivo MidiWriter.cpp continha diversos números mágicos associados a bytes status do protocolo MIDI (`0x90, 0x80, 0xC0, 0xB0`), máscaras de canal (`0x0F`) e índice do controlador de volume (`0x7F`). Além de constantes de tempo de andamento (`60000000, 120`).
- **Ação Corretiva**: Foram criadas constantes simbólicas descritivas em um namespace anônimo no arquivo MidiWriter.cpp (`MIDI_STATUS_NOTE_ON, MIDI_CHANNEL_MASK, MIDI_CC_VOLUME, MICROSECONDS_PER_MINUTE`), eliminando os valores numéricos literais e tornando a representação do protocolo MIDI autodescritiva.

Conclusão

Com a aplicação destas modificações baseadas no relatório de revisão de código da Fase 3, o projeto atingiu um nível de maturidade superior, garantindo o isolamento da lógica de negócios em relação à interface visual, a proteção do estado interno das vozes e a eliminação de redundâncias, respeitando inteiramente os princípios de design de software orientado a objetos SOLID e as especificações.